Virtual Machines

Why Virtualize?

- **Consolidate machines**
  - Huge energy, maintenance, and management savings
- **Isolate performance, security, and configuration**
  - Stronger than process-based
- **Stay flexible**
  - Rapid provisioning of new services
  - Easy failure/disaster recovery (when used with data replication)
- **Cloud Computing**
  - Huge economies of scale from multiple tenants in large data centers
  - Savings on mgmt, networking, power, maintenance, purchase costs
- **Corporate employees**
  - Can choose own devices

Virtual Machines Background

- **Observation**: instruction-set architectures (ISA) form some of the relatively few well-documented complex interfaces we have in world
  - Machine interface includes meaning of interrupt numbers, programmed I/O, DMA, etc.
- Anything that implements this interface can execute the software for that platform
- A *virtual machine* is a software implementation of this interface (often using the same underlying ISA, but not always)
  - Original paper on whether or not a machine is virtualizable: Gerald J. Popek and Robert P. Goldberg (1974). "Formal Requirements for Virtualizable Third Generation Architectures". *Communications of the ACM* 17 (7): 412 –421.

Many VM Examples

- **IBM initiated VM idea to support legacy binary code**
  - Support an old machine's on a newer machine (e.g., CP-67 on System 360/67)
  - Later supported multiple OS's on one machine (System 370)
- **Apple's Rosetta ran old PowerPC apps on newer x86 Macs**
- **MAME is an emulator for old arcade games (5800+ games!!)**
  - Actually executes the game code straight from a ROM image
- **Modern VM research started with Stanford's Disco project**
  - Ran multiple VM's on large shared-memory multiprocessor (since normal OS's couldn't scale well to lots of CPUs)
- **VMware (founded by Disco creators):**
  - Customer support (many variations/versions on one PC using a VM for each)
  - Web and app hosting (host many independent low-utilization servers on one machine – "server consolidation")

VM Basics

- **The real master is no longer the OS but the "Virtual Machine Monitor" (VMM) or "Hypervisor" (hypervisor > supervisor)**
- **OS no longer runs in most privileged mode (reserved for VMM)**
  - x86 has four privilege "rings" with ring 0 having full access
  - VMM = ring 0, OS = ring 1, app = ring 3
  - x86 rings come from Multics (also x86 segment model)
  - **Hardware-Assisted Virtualization (Modern):** Instead of using Ring 1, modern CPUs introduced a "Root" and "Non-Root" mode. Both the Host and Guest can have their own "Ring 0," but the Guest's Ring 0 is "Non-Root," meaning any sensitive instruction causes a **VM Exit** back to the Host (Root mode).

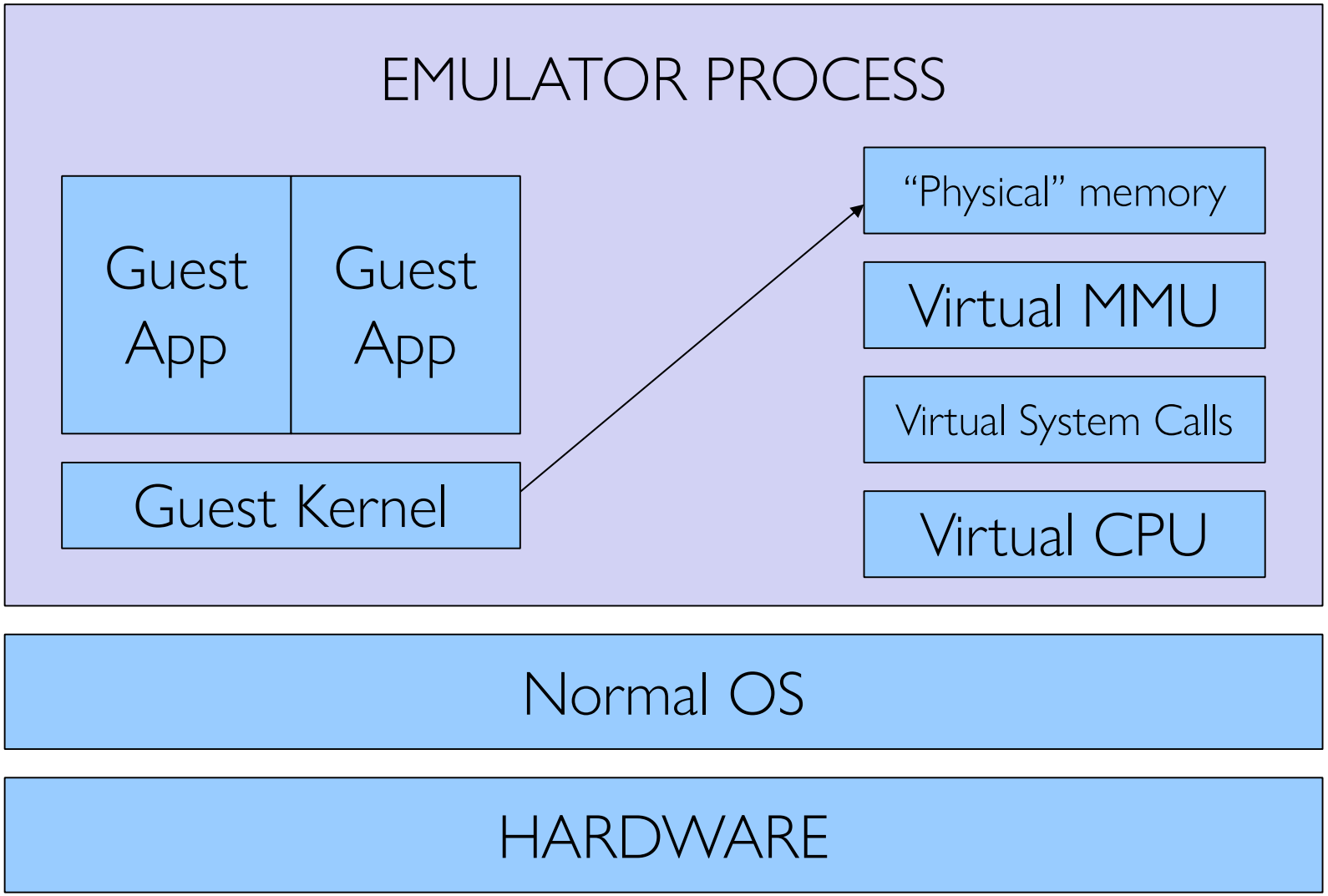
Virtualization Approaches

- **Disco/VMware/IBM: Complete virtualization – runs unmodified OSs and applications**
  - Use software emulation to shadow system data structures,
  - Dynamic binary rewriting of OS code that modifies system structures, or
  - Hardware virtualization support
- **Denali introduced the idea of "paravirtualization" – change interface some to improve VMM performance/simplicity**
  - Must change OS and some apps (e.g., those using segmentation) – easy for Linux, hard for MS (requires their help!)
  - But can support 1,000s of VMs on one machine...
  - Great for web hosting
- **Xen: change OS but not applications – support the full *Application Binary Interface* (ABI)**
  - Faster than a full VM – supports ~100 VMs per machine
  - Moving to a paravirtual VM is essentially porting the software to a very similar machine

Virtualization Approaches

- **Instruction Resolution: Binary Translation vs. VT-x**
- **When a Guest OS calls for a privileged operation (like CLI to clear interrupts or LIDT to load a descriptor table), the hypervisor must intervene:**
  - **Binary Translation:** The VMM scans the Guest's code in real-time. When it sees a privileged instruction, it replaces it with a jump to a "snippet" of code managed by the VMM. This snippet simulates the effect of the instruction on *virtual* hardware without affecting the *physical* hardware.
  - **Direct Execution:** For 99% of code (like calculating values in Excel), the instructions are identical for the CPU regardless of whether they are in a VM or on a host. These run at native speed.

How to Build a VMM 1: SW Emulation (Disco)



I/O Shielding and Device Emulation

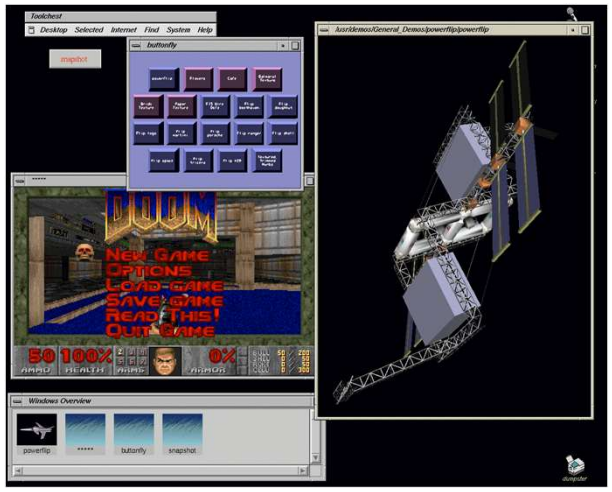
- **VMware "shields" I/O by providing a layer of virtual hardware. The Guest OS doesn't see your specific Realtek Ethernet card; it sees a VMware VMXNET3 adapter.**
- **When the Guest OS sends data to the virtual network card:**
  - The instruction triggers a **Trap** to the VMM.
  - The VMM context-switches to the **VMX process** (the application running on your Windows/Linux host).
  - The VMX process uses standard Host OS APIs (like Win32 or Linux Syscalls) to send that data through the physical hardware.
  - This ensures that a crash in the Guest's driver cannot crash the physical hardware.

System Call Resolution

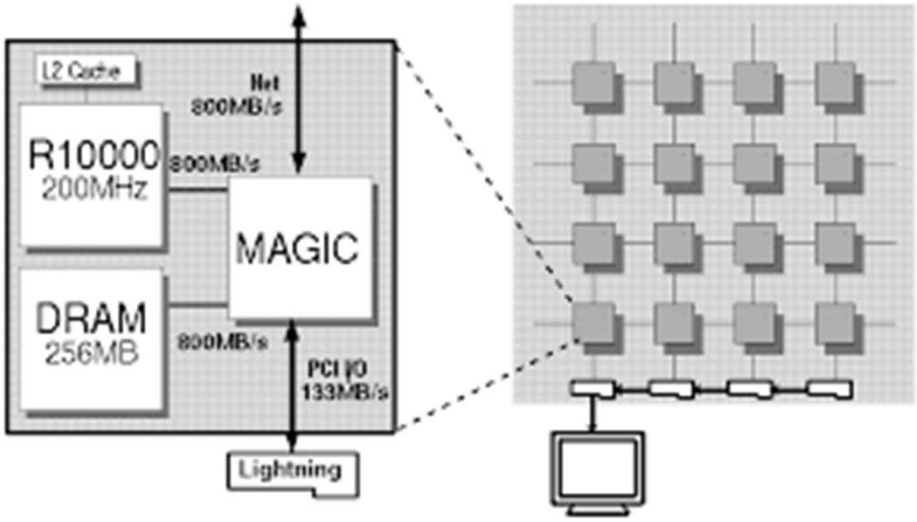
- **It is important to distinguish between two types of "calls":**
  - **Guest App to Guest OS:** These are resolved entirely inside the VM. The VMM simply ensures the transition between Guest Ring 3 and Guest Ring 1/0 happens correctly.
  - **Guest OS to Hardware:** These are "resolved" by the VMM. The VMM maintains a **Shadow Page Table** (or uses **Extended Page Tables - EPT** in hardware). When the Guest OS tries to map memory, the VMM intercepts this and maps the "Guest Physical Address" to the actual "Host Physical Address," ensuring the VM stays within its allocated RAM "sandbox."

Disco

- Extend modern OS to run efficiently on shared memory multiprocessors with minimal OS changes
- A VMM built to run multiple copies of Silicon Graphics IRIX operating system on Stanford Flash multiprocessor

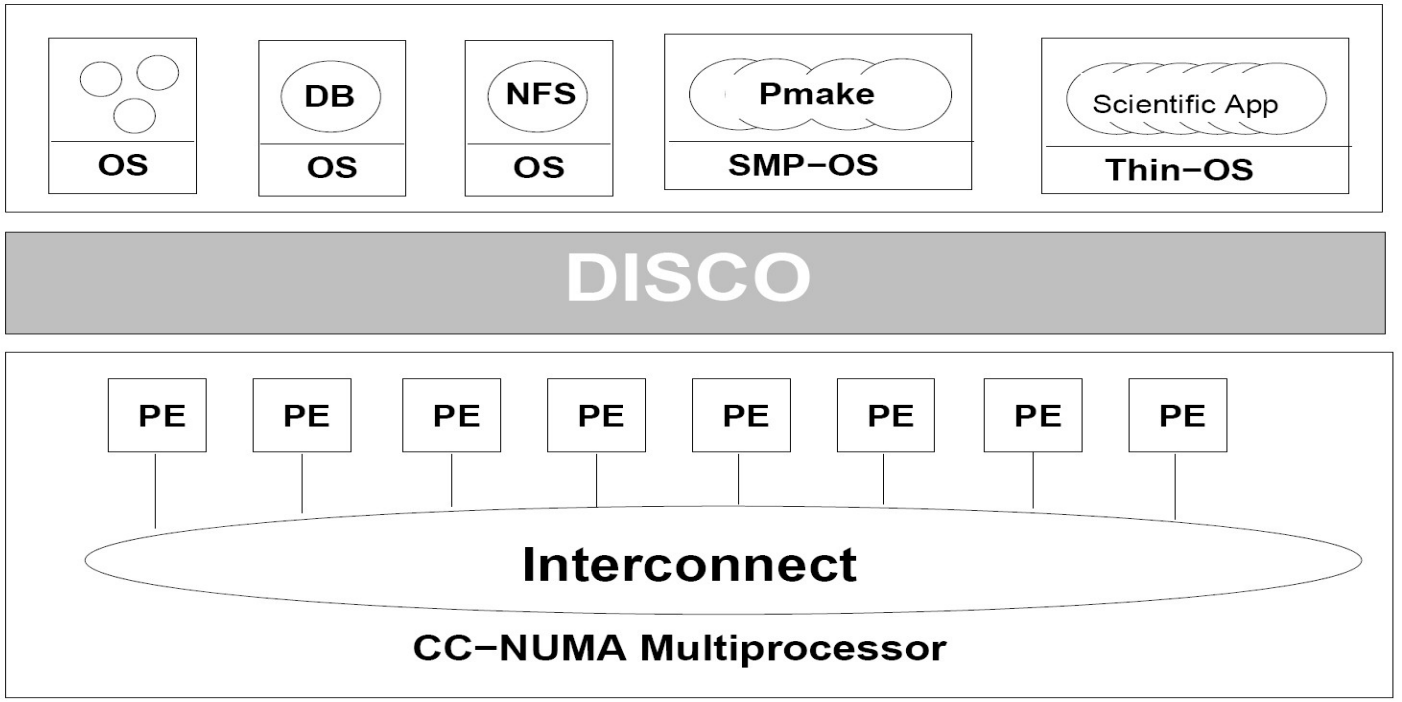


IRIX Unix based OS



Stanford FLASH: cache coherent NUMA

Disco Architecture



Disco's Interface

- **Processors**
  - MIPS R10000 processor: emulates all instructions, MMU, trap architecture
  - Extension to support common processor operations
    - Enabling/disabling interrupts, accessing privileged registers
- **Physical memory**
  - Contiguous, starting at address 0
- **I/O devices**
  - Virtualize devices like I/O, disks
  - Physical devices multiplexed by Disco
  - Special abstractions for SCSI disks and network interfaces
    - Virtual disks for VMs
    - Virtual subnet across all virtual machines

Disco Implementation

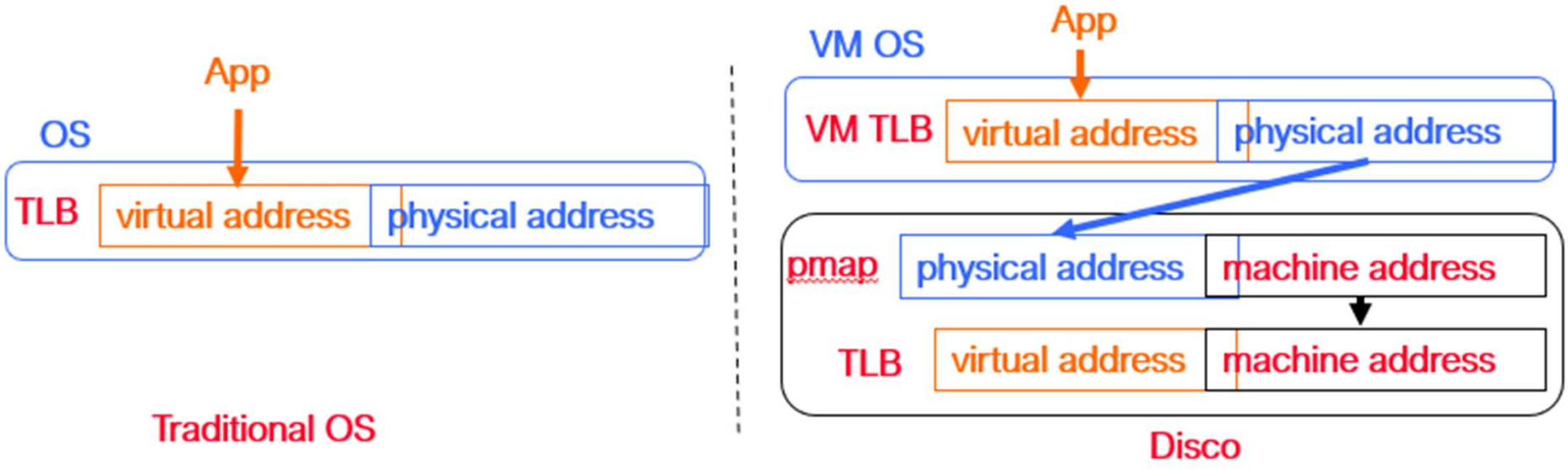
- Multi threaded shared memory program
- Attention to NUMA memory placement, cache aware data structures and IPC patterns
- Disco code copied to each flash processor
- Communicate using shared memory

Virtual CPUs

- **Direct execution on real CPU:**
  - Set real CPU registers to those of virtual CPU/Jump to current PC of VCPU
  - Privileged instructions must be emulated, since we won't run the OS in privileged mode
  - Disco runs privileged, OS runs supervisor mode, apps in user mode
- **An OS privileged instruction causes a trap which causes Disco to emulated the intended instruction**
- **Maintains data structure for each virtual CPU for trap emulation**
  - Trap examples: page faults, system calls, bus errors
- **Scheduler multiplexes virtual CPU on real processor**

Virtual Physical Memory

- **Extra level of indirection: physical-to-machine address mappings**
  - VM sees contiguous physical addresses starting from 0
  - Map physical addresses to the 40 bit machine addresses used by FLASH
  - When OS inserts a virtual-to-physical mapping into TLB, translates the physical address into corresponding machine address



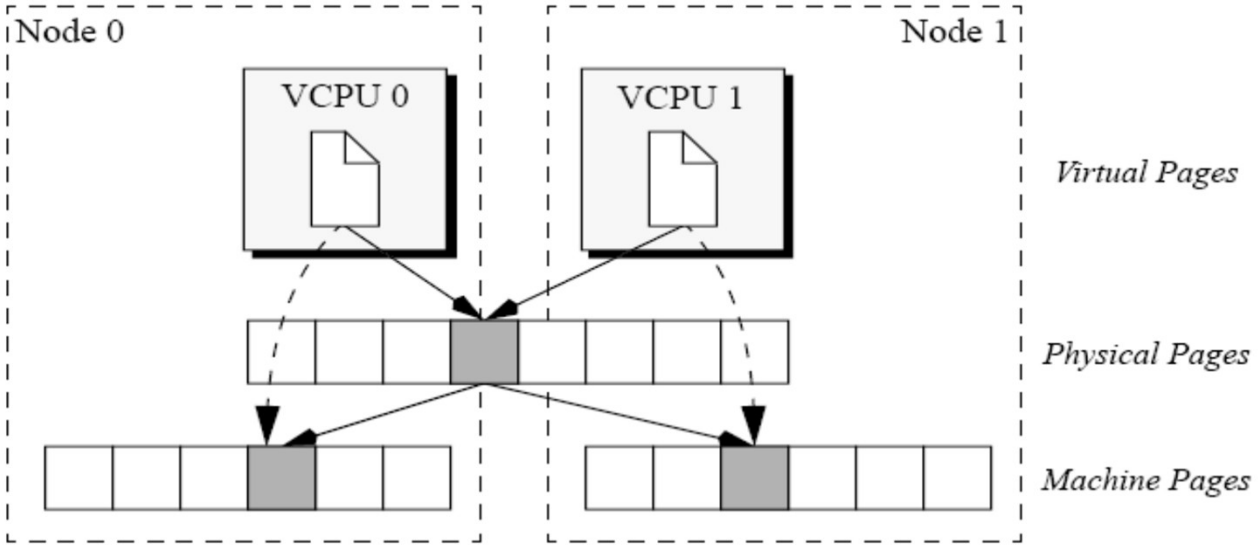
Virtual Physical Memory

- **Extra level of indirection: physical-to-machine address mappings**
  - VM sees contiguous physical addresses starting from 0
  - Map physical addresses to the 40 bit machine addresses used by FLASH
  - When OS inserts a virtual-to-physical mapping into TLB, translates the physical address into corresponding machine address
  - To quickly compute corrected TLB entry, Disco keeps a per VM pmap data structure that contains one entry per VM physical page
  - Each entry in TLB tagged with address space identifier (ASID) to avoid flushing TLB on MMU context switches (within VM)
- **Must flush the real TLB on VM switch**
- **Somewhat slower:**
  - OS now has TLB misses (not direct mapped)
  - TLB flushes are frequent
  - TLB instructions are now emulated
- **Disco maintains a second-level cache of TLB entries:**
  - This makes the VTLB seem larger than a regular R10000 TLB
  - Disco can absorb many TLB faults without passing them through to the real OS

NUMA Memory Management

- **Dynamic Page Migration and Replication**
  - Pages frequently accessed by one node are migrated
  - Read-shared pages are replicated among all nodes
  - Write-shared are not moved, since maintaining consistency requires remote access anyway
  - Migration and replacement policy is driven by cache-miss-counting facility provided by the FLASH hardware

Transparent Page Replication



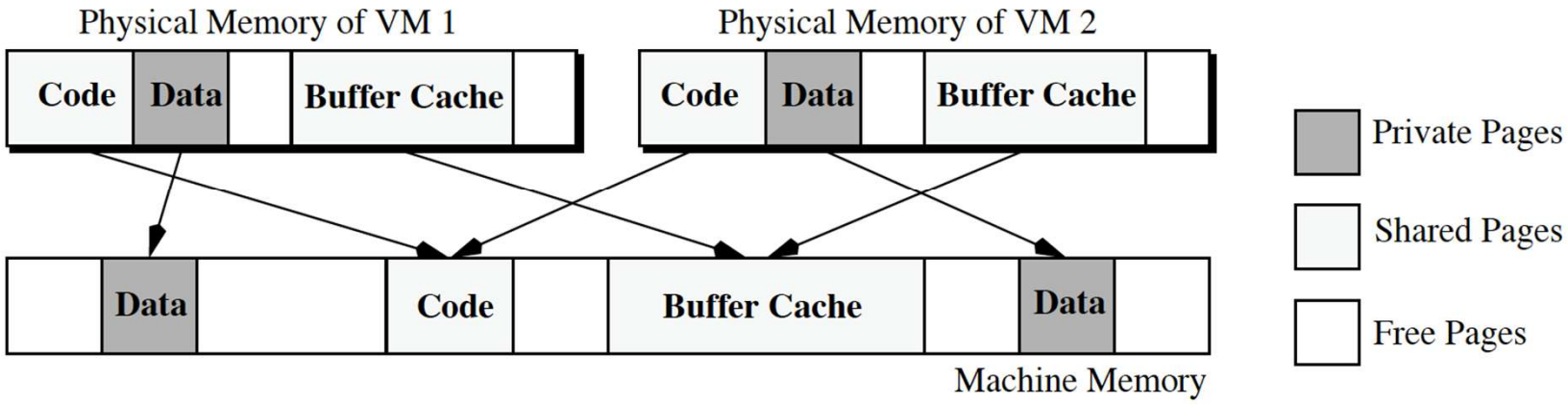
1. Two different virtual processors of same virtual machine logically read-share same physical page, but each virtual processor accesses a local copy
2. *memmap* maintains an entry for each machine page that contains which virtual page reference it; used during TLB shootdown*

*Processors flushing their TLBs when another processor restrict access to a shared page.

I/O (disk and network)

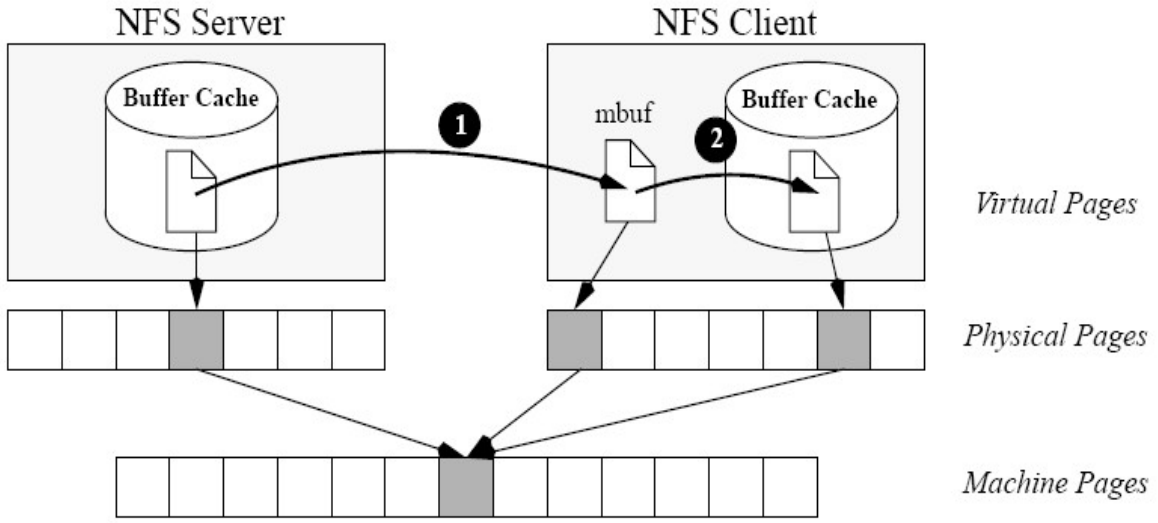
- **Emulated all programmed I/O instructions**
- **Can also use special Disco-aware device drivers (simpler)**
- **Main task: translate all I/O instructions from using PM addresses to MM addresses**
- **Optimizations:**
  - Larger TLB
  - Copy-on-write disk blocks
    - Track which blocks already in memory
    - When possible, reuse these pages by marking all versions read-only and using copy-on-write if they are modified
    - => shared OS pages and shared executables can really be shared.
- **Zero-copy networking along fake "subnet" that connect VMs within an SMP**
  - Sender and receiver can use the same buffer (copy on write)

Transparent Page Sharing



- **Global buffer cache that can be transparently shared between virtual machines**

Transparent Page Sharing over NFS



1. **The monitor's networking device remaps data page from source's machine address to destination's**
2. **The monitor remaps data page from driver's to client's buffer cache**

Impact

- **Revived VMs for the next 20 years**
  - Now VMs are commodity
  - Every cloud provider, and virtually every enterprise uses VMs today
- **VMWare successful commercialization of this work**
  - Founded by authors of Disco in 1998, $6B revenue today
  - Initially targeted developers
  - Killer application: workload consolidation and infrastructure management in enterprises

Summary

- **Disco VMM hides NUMA-ness from non-NUMA aware OS**
- **Disco VMM is low effort**
  - Only 13K LoC
- **Moderate overhead due to virtualization**
  - Only 16% overhead for uniprocessor workloads
  - System with eight virtual machines can run some workloads 40% faster

How to Build a VMM: Paravirtualization (Xen)

Q. But when is this a safe alteration?

A. Let the humans worry about it

- **Manually hack the OS: "paravirtualization"**